

Accelerating Programmable Electrical Rules Checking (PERC) in Physical Design

A White Paper on Reliability Signoff Bottlenecks,
Synthetic Modeling, and Runtime Optimization Strategies

Physical Design Algorithms Implementation

Companion artifact: C++17 PERC Runtime Optimizer project

August 2026

Abstract. Programmable Electrical Rules Checking (PERC) verifies integrated-circuit reliability constraints—especially electrostatic discharge (ESD) related rules—that neither design-rule checking (DRC) nor layout-versus-schematic (LVS) can fully express. In modern SoC signoff, PERC wall-clock time is dominated less by topology identification itself than by parasitic resistance extraction and subsequent point-to-point (P2P) resistance and current-density (CD) analysis, often repeated after every engineering change order (ECO). This white paper explains the industrial PERC flow, clarifies cost centers (including metal fill and RC extraction upstream of PERC), and presents an educational C++17 framework that models netlist-level PERC checks and graph-based P2P/CD solves on synthetic hierarchical designs. We evaluate baseline, region-of-interest (ROI), hierarchical, parallel, incremental, and combined optimized engines, and discuss how the measured levers map to production reliability platforms such as Siemens Calibre PERC and Synopsys IC Validator PERC.

Contents

1. Introduction
 2. Background: PERC in the Physical Verification Stack
 3. Anatomy of an ESD-Oriented PERC Flow
 4. Where Runtime Actually Goes
 5. Upstream Cost Centers: Metal Fill and RC Extraction
 6. The PERC Runtime Optimizer Project
 7. Synthetic Data Model (What It Is — and Is Not)
 8. Implemented Checks and Engines
 9. Experimental Methodology and Results
 10. Interpretation and Guidance for Practitioners
 11. Limitations and Threats to Validity
 12. Future Work
 13. Conclusion
- References
- Appendix A. Repository Layout and Reproducibility

1. Introduction

As process nodes advance and SoC integration grows, electrical reliability constraints have become first-class signoff requirements alongside timing, power, and geometric DRC. Foundries and design houses encode ESD, electrical overstress (EOS), multi-domain voltage rules, and related methodology checks in programmable rule decks. The resulting verification class is widely known as **Programmable Electrical Rules Checking (PERC)**.

Unlike classical DRC (geometry-centric) and LVS (device/net correspondence), PERC combines *connectivity intent* with, for several check families, *layout parasitics*. That hybrid nature makes PERC powerful—and expensive. Design teams routinely report multi-hour to multi-day turnaround for full-chip reliability regressions, especially when ECOs force repeated extraction and rechecking.

This white paper has three goals:

- **Explain** PERC’s role, check families, and industrial data flow in precise terms.
- **Disambiguate** expensive operations: ESD path identification versus P2P/CD, and upstream metal-fill / RC-extraction costs that are often conflated with “PERC runtime.”
- **Document** an open C++17 educational project that implements PERC-like checks and runtime optimizations on synthetic netlists, with measured results and clear limitations.

2. Background: PERC in the Physical Verification Stack

2.1 What PERC is

PERC is a method for checking reliability issues of IC designs that cannot be checked with DRC or LVS alone. Rules involve connectivity and netlist information and must be customizable from design to design—hence *programmable*. Commercial platforms include Siemens Calibre PERC and Synopsys IC Validator PERC.

2.2 The four check families

Family	Primary inputs	Typical intent	Dominant cost
Netlist checks	Schematic or LVS-extracted netlist	Clamp presence, floating gates, EOS / level-shifter topology, domain rules	Graph traversal / rule evaluation over large netlists
Netlist-driven layout (LDL/NDL)	Netlist + GDS/OASIS	Voltage-aware spacing and geometry on connectivity-selected regions	ROI identification + geometry operations
Current density (CD)	Layout + R-extraction + assumed ESD current	Metal can carry discharge current without EM / thermal failure	Parasitic R mesh construction and path/current analysis
Point-to-point (P2P) resistance	Layout + R-extraction	Discharge path resistance low enough that current prefers the clamp path	Many source→sink resistance queries on resistor networks

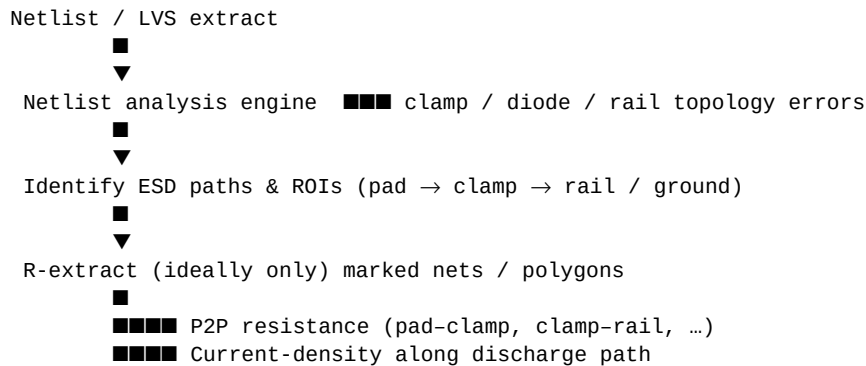
Table 1. PERC check families and primary cost drivers.

2.3 Placement in the signoff timeline

PERC typically runs after the design is LVS-clean (for layout-aware checks), though schematic-only netlist checks can start earlier. It sits alongside DRC, LVS, fill, extraction, timing, and IR/EM signoff. Because reliability failures can escape functional test patterns, PERC is treated as a gate for tapeout on many products (especially automotive, industrial, and high-reliability segments).

3. Anatomy of an ESD-Oriented PERC Flow

ESD applications exercise all four check families. A representative flow is:



The netlist analysis engine is the programmable core: it decides which structures exist, which paths are critical, and which layout regions must be examined. Layout-heavy checks then depend on parasitic resistance models of those regions.

4. Where Runtime Actually Goes

4.1 The common misconception

A frequent question is whether **ESD path identification** or **P2P/CD checking** dominates runtime. In production flows, path identification is usually a comparatively cheap connectivity/graph analysis step. The expensive work is almost always:

- Building or updating a parasitic **R** (and often C) network for relevant metals, and
- Solving many **P2P** queries and **CD** path/current analyses on that network.

Path identification still matters enormously for *scoping*: a poor ROI causes over-extraction and over-checking; a correct ROI is what makes LDL-style acceleration possible.

4.2 Scaling pressures at SoC size

- Millions to billions of devices/nets in the connectivity graph.
- Hundreds to thousands of IO pads, each inducing multiple path endpoints.
- Naïve flows that flatten hierarchy and re-extract the full chip after every ECO.
- Deep, context-aware rule decks (voltage propagation, multi-power domains).

5. Upstream Cost Centers: Metal Fill and RC Extraction

When engineers say “PERC is slow,” they often measure an umbrella regression that includes steps that are logically upstream of the PERC rule engine. Two of the most important are metal fill and parasitic extraction.

5.1 Metal fill (dummy metal)

Dummy metal fill is inserted to satisfy density and manufacturing rules. It is geometry-heavy: large numbers of fill shapes are created, legalized, and verified. Fill can dominate physical-verification runtime on its own and also **changes parasitics**, so extraction and P2P/CD results are fill-dependent. Fill is therefore both a direct runtime

cost and an indirect multiplier on PERC-related extract/check cost.

5.2 R and C extraction

Tools such as StarRC (and equivalents) build resistive (and capacitive) models of interconnect from layout. For ESD P2P/CD, resistance accuracy on discharge paths is critical. Full-chip extract is often one of the largest wall-clock components in the reliability loop. Best practice is netlist-driven / ROI extract: mark ESD-critical nets and extract only what P2P/CD need—subject to accuracy requirements.

5.3 How this paper’s artifact relates

Important clarification: the companion C++ project does *not* implement metal fill or a layout extractor. It assumes an already-available resistive model (an abstract graph) and focuses on check/solve and reuse strategies. Section 7 details the synthetic data model so this boundary is unambiguous.

Industrial step	Modeled in C++ project?	Notes
GDS/OASIS layout	No	No polygons, layers, or fill geometries
Metal fill insertion	No	Acknowledged as major real-world cost
RC extraction (StarRC-class)	Abstracted	Pre-baked weighted RGraph edges
Netlist PERC rules	Yes	Clamps, floating gates, etc.
P2P / CD solves	Yes (graph)	Dijkstra / path walks on RGraph
Incremental metadata reuse	Yes	Scope fingerprints + ECO fast path

Table 2. Industrial steps versus project coverage.

6. The PERC Runtime Optimizer Project

The repository project **PERC Runtime Optimizer** is a C++17 educational framework. Its purpose is not to replace Calibre or IC Validator, but to make PERC’s algorithmic bottlenecks tangible: one can generate a hierarchical synthetic design, run baseline checks, and compare optimized engines with wall-clock measurements.

6.1 Design goals

- Faithful *structure* of PERC stages (netlist analysis → ROI → P2P/CD).
- Measurable optimization levers used in industry: ROI pruning, hierarchy, parallelism, incremental reuse.
- Zero dependency on proprietary runsets or licensed layout databases.
- Reproducible CLI benchmarks and a small unit-test suite.

6.2 Non-goals

- Foundry signoff accuracy or rule-deck compatibility.
- GDS parsing, DRC, LVS, or fill engines.
- Full field-solver or SPEF/DSPF-quality extraction.

7. Synthetic Data Model (What It Is — and Is Not)

7.1 Not a GDS

The synthetic stimulus is **not** a GDS or OASIS file. There is no dummy-metal-filled layout. Instead, `generate_design()` constructs:

- **A hierarchical netlist:** IO pads, optional ESD clamps (some intentionally missing), MOSFETs/diodes per block, local and global supplies, and interface buffers.
- **An abstract resistor graph (RGraph):** undirected weighted edges among nets that stand in for interconnect parasitics that a real extractor would produce.

7.2 Why this abstraction

P2P and CD in industry consume extracted R networks. By generating a resistor graph directly, the project isolates the *query/solve* and *reuse* problems without requiring a layout database. This is appropriate for algorithm study; it is insufficient for predicting absolute industrial runtimes, where fill + extract often dominate.

7.3 ECO mutation

To study incremental checking, `mutate_eco()` touches MOSFET drain nets inside a limited number of blocks (default: one block). Untouched blocks keep stable fingerprints, enabling metadata hits—analogueous to commercial metadata reuse across ECO cycles.

8. Implemented Checks and Engines

8.1 Checks

Check	Rule ID(s)	Method sketch
ESD clamp presence	ESD_CLAMP_MISSING	Every pad net must attach to an ESDClamp device
Floating gates	FLOATING_GATE	MOSFET gates with zero non-gate drivers (indexed writer map)
P2P resistance	P2P_RESISTANCE_HIGH / P2P_PATH_MISSING	Dijkstra shortest resistive path pad→VSS and pad→VDD vs limit
Current density (simplified)	CURRENT_DENSITY	Along pad→VSS shortest path, flag high I·R edge stress proxy

Table 3. Implemented PERC-like checks.

8.2 Engines

Engine	Strategy
baseline	Full-chip: all rules, every time
roi	Prune floating-gate scope toward ESD-relevant devices
hierarchical	Chip-level ESD/P2P/CD; per-block FG with shared writer index
parallel	std::async workers over pad-pair P2P/CD slices
incremental	MetadataStore keyed by scope fingerprint; ECO skips untouched blocks
optimized	ESD-stable chip cache + parallel P2P/CD + hierarchical FG reuse

Table 4. Runtime engines.

8.3 Metadata and fingerprints

Incremental reuse stores per-scope results keyed by a structural fingerprint. Chip-level ESD/P2P/CD uses an **ESD-stable fingerprint** (pads, rails, clamp/IO devices and pad-rail R edges) so core-logic ECOs do not spuriously invalidate chip ESD results. Block scopes fingerprint local nets/devices. When `touched_blocks` is known, untouched blocks reuse the latest cached result without re-hashing.

9. Experimental Methodology and Results

9.1 Setup

Measurements below were taken from the project CLI on a Linux environment using the default `--bench` configuration unless noted: 256 pads, 16 blocks, 500 devices/block (~8.8k devices, ~16.7k nets, ~22.8k R-edges), compiled with `g++ -O2 -pthread`. Absolute times are machine-specific; ratios are the intended takeaway.

9.2 Stage-level profile (baseline internals)

Instrumenting individual checks on the bench design (averaged) yields the approximate breakdown in Table 5. In this *extract-free* model, netlist floating-gate scanning and graph P2P/CD are both visible; ESD ROI identification remains a minority cost—consistent with the industrial claim that path ID is not the primary bottleneck once extraction exists.

Stage	Avg time (s)	Share
ESD ROI / path ID (net scan)	0.0010	~9%
ESD clamp netlist check	0.0001	~1%
Floating-gate netlist check	0.0056	~47%
P2P resistance (Dijkstra)	0.0032	~27%
Current-density path walks	0.0018	~15%
Total (sum of stages)	0.0118	100%

Table 5. Stage profile on synthetic bench design (no layout extract).

Note: floating-gate share is inflated by the synthetic stimulus, which intentionally leaves many gate nets without drivers to stress the checker. In real netlists, FG cost is typically smaller relative to `extract+P2P/CD`.

9.3 Engine comparison

Engine	Time (s)	Observations
baseline	0.0120	Full recompute reference
roi	0.0141	Similar quality; prune overhead can outweigh savings at this size
hierarchical	0.0116	Shared FG index; comparable to baseline cold
parallel	0.0079	~1.5× vs baseline on pad-pair heavy work (4 workers)
incremental (cold)	0.0746	Fingerprint+populate cache costs more cold
incremental (warm)	0.0007	~100× vs cold incremental; ■ more cold
incremental (warm)	0.0007	~100× vs cold incremental; ■ baseline when unchanged

optimized (warm)	0.0007	Same warm-cache benefit
------------------	--------	-------------------------

Table 6. Engine wall-clock on unchanged design (bench config).

9.4 ECO incremental recheck

After a 5% MOSFET-drain ECO confined to a single block:

Engine	Time (s)	Cache behavior
baseline	0.0104	Full recompute
incremental	0.0034	16 hits / 1 miss (recheck touched block only)
optimized	0.0031	Same reuse pattern + parallel chip path when needed

Table 7. Post-ECO recheck (one touched block).

Violation counts matched between baseline and incremental/optimized after ECO (7449 total), indicating the fast path did not drop the newly introduced floating-gate effect in the touched block.

10. Interpretation and Guidance for Practitioners

Even though absolute times in the toy model are milliseconds, the **shape** of the results matches industrial practice:

- **Do not over-invest in speeding path ID alone** if extract+P2P/CD dominate your traces.
- **Invest in ROI / LDL:** mark ESD nets early; extract and check only what those paths need.
- **Treat ECO as the common case:** metadata reuse and hierarchical invalidation beat heroic cold-run constant-factor tuning.
- **Parallelize embarrassingly partitioned P2P pairs** across pads/domains once the R model exists.
- **Account for fill and extract in program plans:** a “PERC project” that ignores them will miss the real critical path.

For teams building internal accelerators or research prototypes, a useful layering is: (1) connectivity/ROI engine, (2) extract subset manager, (3) P2P/CD solver farm, (4) persistent metadata store keyed by hierarchical scopes.

11. Limitations and Threats to Validity

- **No layout database:** cannot reproduce geometry-limited LDL or fill interactions.
- **No real extractor:** RGraph edges are synthetic; Dijkstra on a modest mesh understates industrial solve cost.
- **Simplified CD:** I·R proxy is pedagogical, not a current-density field model.
- **Synthetic FG population:** skews stage profiles versus production netlists.
- **Single-machine pthread scaling:** does not model distributed farm / license / disk bottlenecks.
- **Not signoff-equivalent:** results must not be used as reliability certification evidence.

12. Future Work

- **Synthetic layout + fill model:** tile-based metals with dummy fill insertion timed separately from checks.
- **Extract emulator:** build R (and C) from the synthetic layout with configurable accuracy/runtime tradeoffs.

- **Richer ESD topologies:** secondary clamps, rail clamps, diode chains, multi-domain crossers.
- **Voltage-aware LDL checks:** propagate domain voltages and trigger geometry queries on ROIs.
- **Persistent on-disk metadata** and hierarchical invalidation integrated with a mock P&R ECO stream.
- **GPU/batch shortest paths** for large pad-pair sets on huge resistor meshes.

13. Conclusion

PERC is essential for catching ESD/EOS-class reliability issues outside the reach of DRC and LVS. Its runtime pain is real, but it is easy to mis-attribute. ESD path identification is necessary scaffolding; the dominant costs in production are typically parasitic extraction and P2P/CD analysis—often amplified by metal fill and by full-chip redo after ECO.

The C++17 PERC Runtime Optimizer makes these ideas concrete with synthetic hierarchical netlists, graph-based P2P/CD, and engines that demonstrate ROI pruning, hierarchy, parallelism, and incremental metadata reuse. Warm-cache and ECO-local recheck show order-of-magnitude or multi-fold speedups in the model, mirroring the strategic importance of reuse in commercial reliability platforms. Extending the artifact toward fill and extraction would close the largest remaining gap between educational measurement and industrial wall-clock reality.

References

- [1] Synopsys, “What is PERC (Programmable Electrical Rules Checking)?” Synopsys Glossary. <https://www.synopsys.com/glossary/what-is-programmable-electrical-rules-checking.html>
- [2] Synopsys, IC Validator Physical Verification Datasheet (PERC / NDC / MMC / CD / P2P capabilities).
- [3] Siemens EDA, “Advanced electrical rule checking in IC reliability verification,” Calibre PERC technical paper.
- [4] Siemens EDA, “Increase productivity by reusing metadata for signoff & ECOs,” Calibre PERC metadata reuse paper.
- [5] eInfochips, “Understanding PERC: Definition and Applications for Reliable Design” (ESD, P2P, CD overview).
- [6] Industry practice notes on logic-driven layout (LDL), StarRC-class R-extraction for ESD paths, and dummy metal fill density flows (foundry design manuals; tool-specific user guides).

Appendix A. Repository Layout and Reproducibility

The project lives under `PERC Runtime Optimizer/` with headers in `include/`, sources in `src/`, tests in `tests/`, and concept notes in `docs/`.

```
cd "PERC Runtime Optimizer"
make -j
make test
./perc_optimizer --bench --eco --workers $(nproc)
# regenerate this PDF:
python3 docs/generate_whitepaper.py
```

Primary sources: `design.*` (netlist/RGraph), `generator.*` (synthetic design + ECO), `checks.*` (rules), `engine.*` (baseline/optimized runners), `metadata.*` (cache).

This white paper is intended as an educational and engineering companion document for the open repository. It is not a foundry signoff guide.